

Achieving Microsecond Latencies with Java

How we built a 5M ops/sec exchange in pure Java

Stefan Angelov · Devvxx UK 2026

"Java is too slow for low latency."

Five million ops per second.

0.5 μ s median. p99 $\approx$ 42 μ s.

On a 2014 Xeon.

How small is a microsecond?

Unit	What it is	Real-world analogue
1 ms	one thousandth of a second	a camera flash
1 μs	one millionth of a second	light travels ~300 m
1 ns	one billionth of a second	light travels ~30 cm

A network round-trip across a city: **~5 ms**. A single L1 cache read: **~1 ns**.

Five orders of magnitude. That's the playground we're standing on for the next 30 minutes.

Who needs this kind of latency?

- **Trading & exchanges** — order matching, market data, risk gates
- **Online gaming** — competitive PvP, server-authoritative simulation
- **Real-time bidding** — ad auctions resolved in 10 ms end-to-end
- **Vehicle autopilot & robotics** — sensor fusion, control loops
- **Telco & 5G core** — packet-level processing inside the SLA

“ The techniques are identical across all of these. **The domain is just the budget.** ”

Who am I

- **Stefan Angelov** — Engineering Manager at **Tradu** · Software Architect
- **10+ years** across fintech & gaming — distributed systems, millions of transactions
- **5 years** on the trading-system hot path — matching, risk, market data
- Co-founder of **Hacker4e** — kids' coding academy (Scratch + web)
- Plovdiv, Bulgaria

Why listen: every number on these slides is either from production trading systems I've worked on, or from a JMH benchmark in github.com/cefote/devoxx-uk-2026 — both are open.



What this talk is (and isn't)

Three promises:

1. **Mechanism over recipe** — you'll leave knowing *why*, not just *what*
2. **Every claim runnable** — a JMH benchmark backs every number on every slide
3. **Production-validated** — exchange-core has been doing this for years

Not on the menu: Loom, Valhalla speculation, GraalVM native-image, microbenchmark gotchas. Other talks cover those. This one is about the hot path.

The mental shift

Good Java	Hardware-aware Java
Immutable objects	Pooled, mutable, reused
Dependency injection	Direct calls, JIT-inlinable
Clean abstractions	Hot path is one method
<code>ConcurrentHashMap</code>	One thread owns the data
"GC is fine"	GC is the enemy

“ *"That conversation broke my brain. Everything I thought I knew about writing Java suddenly didn't matter."* ”

Part 1 – The platform

How the JVM actually runs your code

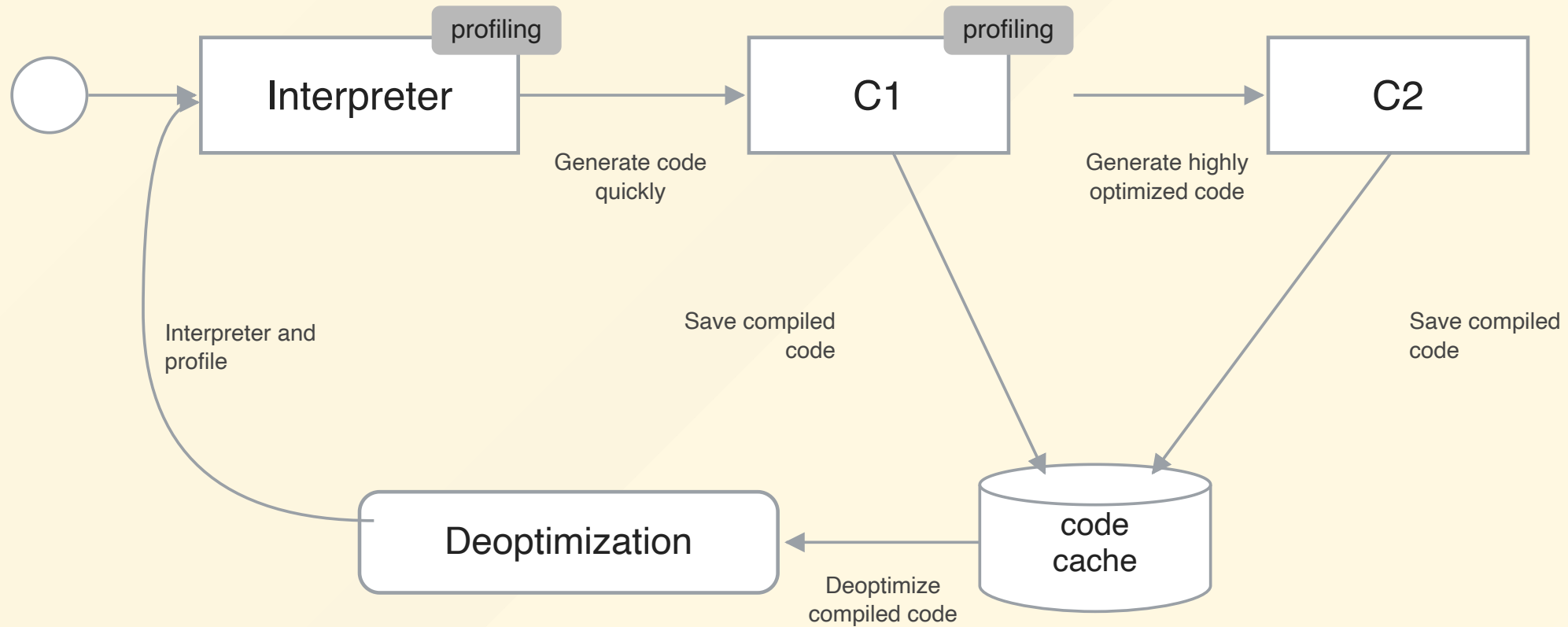
JIT 101 – three tiers, not one

Invocations	Tier	What it does
0 → 1,500	Interpreter	Profile collection – slow but free
1,500 → 10,000	C1	Fast compile, basic optimisations
10,000+	C2	Aggressive, profile-driven native code

After ~10K invocations of a hot method – which happens in **milliseconds** for trading loops – you're running native machine code optimised for *your exact workload*.

Cold-start vs fully-warmed: 10–20× difference. That's why HFT systems warm up before they trade.

The JIT compilation pipeline



What C2 actually does

The JIT knows things you can't:

- **Which branches you take** — 99.9% of the time. Optimises for *those*.
- **Which objects don't escape** — eliminates the allocation entirely (scalar replacement)
- **Which virtual calls are monomorphic** — devirtualises and inlines them
- **Which loops are hot** — unrolls, vectorises, hoists invariants

Inlining is the mother of all optimisations. Three method calls collapse into direct field access. 15–20 ns → 2–3 ns. **7–10× faster.**

The cost of doing nothing

```
// Looks innocent. Allocates a String on every call –  
// even when FINE logging is OFF.  
logger.fine("Welcome to JPrime with " + getNumberOfParticipants() + " participants");  
  
// Same intent. Zero allocation when FINE is disabled.  
if (logger.isLoggable(Level.FINE)) {  
    logger.fine("Welcome to JPrime with {} participants", getNumberOfParticipants());  
}
```

The first version concatenates the string and calls `getNumberOfParticipants()` **every time** — even when nobody is listening. The second short-circuits.

“ One **if**. **Zero allocation in the common path.** The smallest possible example of "lazy beats eager" on the hot path — and the one almost every codebase gets wrong. ”

The 35-byte trap

“ *"I spent two days debugging why our order validation got 10× slower after I extracted a helper method."* ”

The default `MaxInlineSize` is **35 bytes of bytecode**. My "clean" refactor added **8 bytes** to the caller. Pushed it past 35. Killed inlining. **10× slower**.

I reverted the refactoring.

“ Backed by `InliningThresholdBenchmark.java` — the same method, with and without 8 bytes added, run side by side. ”

`-XX:+PrintInlining` – what the JIT actually did

Run the diagnostic flag. Read the output. **Don't guess.**

```
@ 12  com.tradu.OrderBook::match (28 bytes)    inline (hot)
@ 17  com.tradu.OrderBook::priceLevel (15 bytes) inline (hot)
@ 32  com.tradu.OrderBook::matchSlow (180 bytes) too big
@ 41  com.tradu.RiskGate::check (42 bytes)     callee is too large
@ 48  java.util.HashMap::get (28 bytes)        intrinsic
```

Three messages to memorise:

- `inline (hot)` – JIT did the work, you're flying
- `too big` – over `MaxInlineSize` (35 bytes default; `FreqInlineSize` 325 for hot)
- `callee is too large` – same trap, viewed from the caller side

“ The verification loop for every "did my refactor break inlining?" question. ”

Escape analysis – when allocation disappears

Inlining is the gate. Once a method inlines, the JIT asks: **does this object escape?**



Implication: small, short-lived objects can cost zero – **if the calling chain inlines.** Break inlining (the 35-byte trap), the object becomes real again. Allocation returns.

“ Backed by `EscapeAnalysisBenchmark.java`. ”

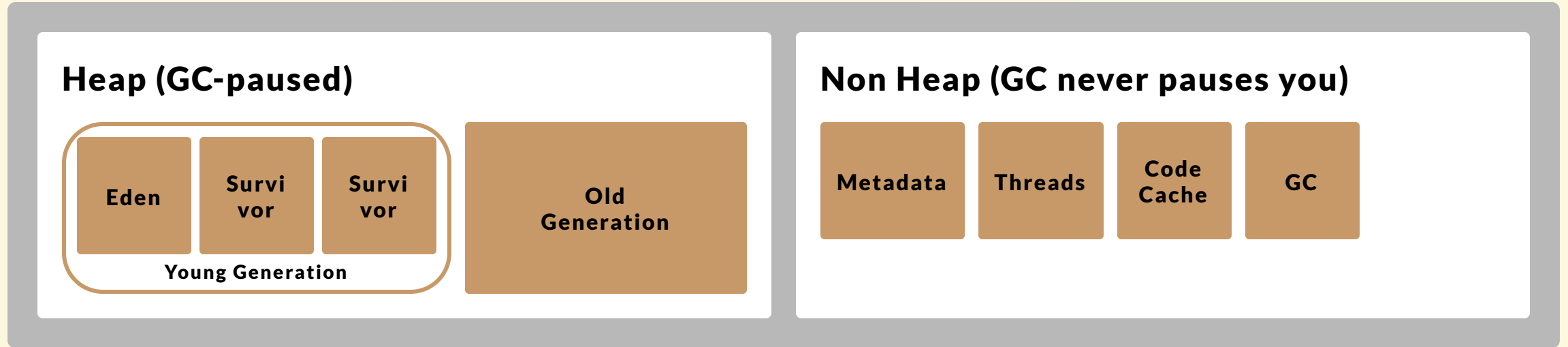
The memory wall

Level	Latency	Cycles	Size
Register	< 1 cycle	1	a handful
L1 cache	~1 ns	3-4	32-64 KB / core
L2 cache	~3 ns	~10	256 KB - 1 MB / core
L3 cache	~12 ns	~40	8-32 MB shared
RAM	~80-100 ns	hundreds	gigabytes
Disk (NVMe)	~100 μ s	millions	terabytes

Going from L1 to RAM is **100×** slower. Going from RAM to disk is **1,000×** slower again.

The hot path lives in cache. Everything else is the slow path.

Java's memory model – heap, stack, and metaspace



Every **new** lands in **Eden**. Every pause comes from cleaning Eden, Survivor, or Old Gen. The Non-Heap region – including the off-heap buffers we'll use in 5 minutes – is **invisible to the GC**.

Why GC kills latency

GC isn't slow on average. **GC is unpredictable.**

Percentile	Pause	Verdict
p50	0.1 ms	fine
p90	1 ms	fine
p99	10 ms	problem
p99.9	50–500 ms	career-ending in trading

One 50 ms pause = 5,000× your latency budget. Backed by `GcImpactBenchmark.java`.

GC pause shapes — picking the collector

Collector	Pause shape	Throughput cost	Heap sweet spot
G1 (<i>default</i>)	10–50 ms typical, 100 ms+ tail	5–10%	4 – 32 GB
ZGC	< 1 ms even at TB scale	10–15%	8 GB – 16 TB
Shenandoah	1–5 ms soft real-time	10–15%	4 GB – 1 TB
Generational ZGC (<i>JDK 21+</i>)	< 1 ms + better throughput	5–10%	4 GB – 16 TB

ZGC and Shenandoah are **concurrent** — they do the work alongside your application threads. Pay in CPU, not in pause time.

Three counter-intuitive truths that everyone in this room was taught the opposite of

Truth #1

Off-heap beats on-heap.

**The garbage collector is a helicopter parent.
If it doesn't know your memory exists,
it can't pause you to clean it up.**

Off-heap mechanism

```
// Allocate ONCE at startup, off the JVM heap  
ByteBuffer direct = ByteBuffer.allocateDirect(1024 * 1024);  
UnsafeBuffer buffer = new UnsafeBuffer(direct);  
  
// Hot path - read/write primitives, zero allocation  
long orderId = buffer.getLong(offset);  
double price = buffer.getDouble(offset + 8);  
int quantity = buffer.getInt(offset + 16);
```

Why off-heap wins

- **No allocation** in the hot path
- **No GC tracking** — the memory isn't on the heap
- **No object headers** — just the bytes you wrote
- **No copy** — same bytes the kernel handed you, parsed in place

Off-heap vs on-heap — the number

15x

On-heap message processing: ~150 ns / message, 3 allocations, GC every ~15 s

Off-heap message processing: ~10 ns / message, **0 allocations, 0 GC**

“ Worst-case (during a young GC): on-heap = 50 ms. Off-heap = 20 ns. **2,500,000× tail-latency improvement.** ”

“ Backed by `OnHeapVsOffHeapMessageBenchmark.java`. Run on the EPYC reference box. ”

When you'd actually do this

Off-heap shines when:

- Hot path processes messages from network or memory-mapped files
- You can pre-size buffers at startup (not in steady state)
- You can give up Java's safety net for raw byte addressing

Off-heap is wrong when:

- You're building a CRUD app at 50 req/s
- Your team can't audit `Unsafe` / `DirectByteBuffer` lifecycle bugs
- The hot path *isn't* the bottleneck

“ Pick one approach and commit. **Mixing on-heap and off-heap in the same hot path is the worst of both worlds.**

Truth #2

Single-threaded beats multi-threaded.

**Locks don't scale.
Adding threads made our account service
800× slower.**

The shard-per-thread pattern

```
// User 100 → shard 0. User 101 → shard 1. User 102 → shard 2.  
// Each shard is a single thread. No locks. Ever.
```

```
public class ShardedAccountService {  
    private final int shardId;  
    private final long shardMask;  
    private final Long2ObjectHashMap<Account> myAccounts;  
  
    public void update(long userId, long delta) {  
        if ((userId & shardMask) != shardId) return;  
        myAccounts.get(userId).balance += delta;  
    }  
}
```

Why sharding wins

- **One thread owns each user.** Forever. No other thread can touch it.
- **No `synchronized`. No `ConcurrentHashMap`. No CAS.**
- **L1 cache stays warm** — the same thread always touches the same data.

Sharded vs contended – the number

100–1000×

3-thread contended `ConcurrentHashMap` + `synchronized` **block**: 150–400 ns/op, p99 in the microseconds

3-thread sharded via Disruptor + Agrona `Long2LongHashMap`: 0.5–2 ns/op, **flat tail**

“ Backed by `SingleThreadedShardVsContendedBenchmark.java`. 3 threads = 3 EPYC physical cores. ”

“ Production proof: exchange-core's matching engine partitions by symbol – same pattern, same outcome. ”

Why locks are this expensive

It's not the lock. It's everything around the lock:

- **Cache-line ping-pong** — the lock word lives in one cache line. Every thread that touches it invalidates the others' copy.
- **Kernel transitions** — contended `synchronized` falls through to OS-level parking. Microsecond-class cost.
- **False sharing on adjacent fields** — two unrelated fields in the same 64-byte line bounce between cores.
- **Coherence traffic** — MESI protocol overhead grows quadratically with core count.

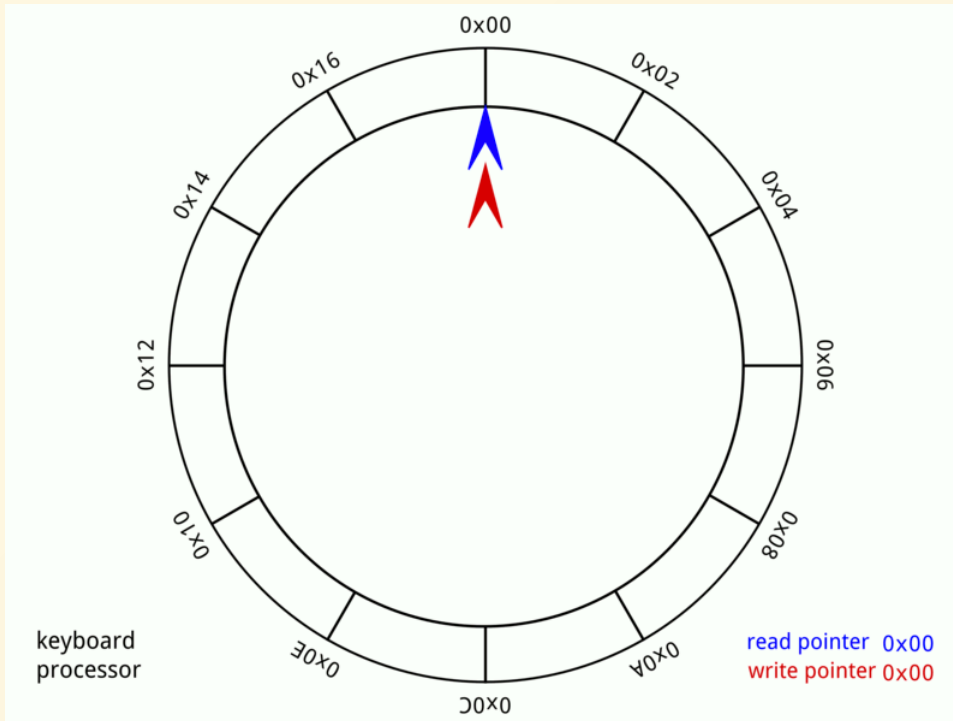
“ Backed by `CasVsLockBenchmark.java` and `FalseSharingBenchmark.java`. ”

Truth #3

Pool beats allocate.

**"Allocation is cheap" — sure, until you do
100,000 of them per second.
That's 240 MB of garbage per minute.**

What's a ring buffer?



A **fixed-size array** with a write cursor that wraps around. Old slots get overwritten by new writes — **no allocation, no resize, no GC**. The exact shape we want for a hot-path object pool.

Ring-buffer object pool

```
public final class RingBufferObjectPool<T> {
    private final T[] pool;
    private final long mask;
    private final AtomicLong cursor = new AtomicLong();

    public RingBufferObjectPool(Supplier<T> factory, int size) {
        if ((size & (size - 1)) != 0) throw new IllegalArgumentException();
        this.pool = (T[]) new Object[size];
        this.mask = size - 1;
        for (int i = 0; i < size; i++) pool[i] = factory.get(); // pre-allocate
    }

    public T acquire() {
        return pool[(int)(cursor.getAndIncrement() & mask)]; // bitwise modulo
    }
    // No release. Objects rotate. Whatever's at index N gets reused on cycle N+size.
}
```

- Pre-allocate at startup. Reuse circularly. **No release method.**
- Size **3–4× peak** — when Bitcoin moves 10%, your order rate triples.

Pooling vs allocation – the number

21×

Allocate-per-order: 6 M allocations / minute, 240 MB garbage, ~20 young GCs

Pooled (1024 objects): 1024 allocations *total*, **0 MB garbage in steady state**

Metric	Allocate	Pooled
p50 latency	45 µs	8 µs (5.6×)
p99 latency	250 µs	12 µs (21×)
GC pauses (60 s window)	~20	0

“ Backed by `ObjectPoolingBenchmark.java`. ”

The pool's evil twins

Canonical objects — for *immutable* values, share the one true instance.

```
private static final Boolean TRUE = Boolean.TRUE;    // never allocate true
private static final BigDecimal ZERO = BigDecimal.ZERO;
```

Lazy initialisation — for objects you might never need, allocate on first use, not in the constructor.

Thread-locals — per-thread instances of stateful, non-thread-safe objects (`SimpleDateFormat`, `NumberFormat`).

Flyweight — share intrinsic state across thousands of logical objects. Same idea, different name.

Part 2 – The libraries

Don't reinvent these.

LMAX Disruptor — the ring buffer that started it all

A bounded, **pre-allocated** ring buffer with **lock-free** producer/consumer coordination via `Sequence` and `SequenceBarrier`.

- **No allocation in steady state** — events are pre-allocated and overwritten
- **Mechanical sympathy** — padded sequences, cache-line aligned
- **Multi-producer, multi-consumer** with explicit dependency graphs
- **Wait strategies** — busy-spin, yield, block — pick your CPU/latency trade-off

“ Used in the original LMAX exchange. Used in exchange-core. Used everywhere a `BlockingQueue` would have been used in 2010. ”

Disruptor vs `ArrayBlockingQueue` – the number

5–15x

`ArrayBlockingQueue` – locks under contention, allocates iterator on `take()`, pauses on capacity wait.

Disruptor SPSC – busy-spin, zero allocation, single-digit nanosecond hand-off.

“ Backed by `DisruptorVsArrayBlockingQueueBenchmark.java`. ”

“ **Caveat:** Disruptor's busy-spin wait strategy burns CPU. It's the right call when latency is the priority. It's the wrong call on a battery-powered device. ”

Chronicle Queue — persistent, microsecond IPC

```
try (ChronicleQueue queue = ChronicleQueue.single("/tmp/queue");
     ExcerptAppender appender = queue.acquireAppender()) {
    appender.writeBytes(b -> b.writeLong(orderId).writeDouble(price));
}
```

- **Memory-mapped file** — between JVMs, between processes, even across reboots
 - **Sub- μ s append** for small messages (16–64 bytes)
 - **Disk-backed** — you get durability essentially for free
 - **No GC** — bytes go straight to the mapped region
- “ If you've ever been tempted to use Kafka for in-host process-to-process messaging — read about Chronicle Queue first. ”

Chronicle Queue – the number

< 1 μ s

Single-thread, single-queue: 16-byte messages, **median $\approx$ 800 ns**. p99 $\approx$ 4 μ s at the published numbers from Chronicle Software. [VERIFY against our run]

Comparable Kafka producer (local broker): 2–10 ms end-to-end, several orders of magnitude higher.

“ Backed by `ChronicleQueueBenchmark.java`. Output dir:
`/tmp/devoxx-uk-bench-queue` (deleted in `@TearDown`). ”

Chronicle Map — off-heap K/V at scale

```
ChronicleMap<Long, Account> accounts = ChronicleMap  
    .of(Long.class, Account.class)  
    .name("accounts")  
    .entries(10_000_000)  
    .averageValueSize(128)  
    .create();
```

- **Off-heap** — millions of entries with **zero GC pressure**
- **Memory-mapped** — survives JVM restart if you back it with a file
- **Concurrent reads, lock-stripped writes** — designed for multi-threaded workloads
- **ConcurrentHashMap replacement** when entry count exceeds a few million

Chronicle Map vs `ConcurrentHashMap` – the number

~ 0 GC

`ConcurrentHashMap` @ 10M entries: GC mark traverses all entries, full GCs of 100–500 ms become regular events.

Chronicle Map @ 10M entries: off-heap, **GC ignores it entirely**, lookup latency stays in tens of nanoseconds.

“ Backed by `ChronicleMapVsConcurrentHashMapBenchmark.java`. ”

Agrona – the foundation

The library you're already using if you've ever touched Disruptor or Aeron.

- `Long2ObjectHashMap`, `Long2LongHashMap` – primitive keys, no boxing, no `Long` cache pollution
- `UnsafeBuffer` – the off-heap buffer everyone else builds on
- `OneToOneRingBuffer` – Aeron's lock-free ring buffer, simpler than Disruptor for single producer / single consumer
- `IdleStrategy` – `BusySpin`, `Backoff`, `Yielding`, `Sleeping` – composable wait strategies

“ If you remember one library name from this talk, **remember Agrona.**

”

Agrona – the numbers

3–6×

`HashMap<Long, Long>` (**boxing!**): Long objects per put/get, allocates `Long` instances, terrible cache behaviour

`Long2LongHashMap` (**Agrona**): primitives, contiguous arrays, **3–6× faster, 0% boxing**

Agrona ring buffer (SPSC) vs `ArrayBlockingQueue`: 3–8× lower latency at single-producer/single-consumer.

“ Backed by `PrimitiveCollectionsBenchmark.java` and `AgronaRingBufferBenchmark.java`. ”

Part 3 – Mechanical sympathy

What the CPU actually wants from you

False sharing – the silent killer

Two unrelated `long` fields. Two threads. One cache line.
Each write invalidates the *other thread's* copy of the line. **Both threads stall.**

```
class DataHolder {  
    public volatile long l1;  
    public volatile long l2;  
}  
  
class DataHolder {  
    @Contended public volatile long l1;  
    @Contended public volatile long l2;  
}
```

Without `@Contended`: the two longs share a cache line. Two threads writing to them ping-pong the line.

With `@Contended`: each long sits alone in its own 64-byte line. **3–8× faster.**

“ Backed by `FalseSharingBenchmark.java`. Requires `-XX:-RestrictContended`. ”

Branch prediction — predictable beats random

```
// Sorted input: branch is predictable. CPU pipeline stays full.  
for (int v : sortedArray) if (v >= 128) sum += v;  
  
// Same code, shuffled input: branch is random. Pipeline stalls on every miss.  
for (int v : shuffledArray) if (v >= 128) sum += v;
```

Same code. Same data. Same comparisons. The sorted version is **2–5× faster** because the CPU's branch predictor has a pattern to lock onto.

Lesson: sort hot-path inputs when you can. Avoid data-dependent branches in tight loops. Pre-compute a sentinel value if it lets you delete an `if`.

“ Backed by `BranchPredictionBenchmark.java`. ”

Thread affinity — pin the thread to the core

```
try (AffinityLock lock = AffinityLock.acquireCore()) {  
    processOrdersForShard0();  
}
```

Combined with **Linux** `isolcpus=1-4` in GRUB:

- Linux scheduler ignores cores 1-4 entirely
- L1/L2 caches stay hot — no migration
- p99 spikes from scheduler migration **disappear**

“ **Linux only.** On macOS the affinity calls are no-ops; the benchmark `@Setup` skips itself with `Assume.assumeTrue(...)`. ”

“ Backed by `ThreadAffinityBenchmark.java`. ”

A few more things you should know exist

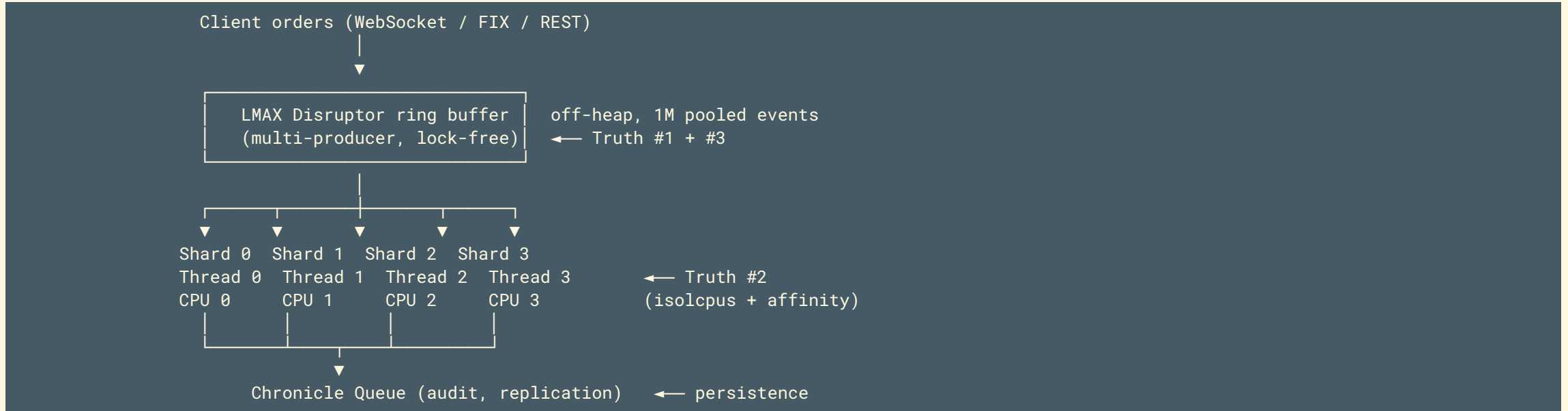
- **Escape analysis** can eliminate allocations entirely — `EscapeAnalysisBenchmark.java`
- **Lock-free CAS** beats `synchronized` under contention by 2–10× — `CasVsLockBenchmark.java`
- **Inlining threshold** is the silent reason your "clean" code regressed — `InliningThresholdBenchmark.java`
- **GC choice matters** — G1 vs ZGC vs Shenandoah have different pause shapes — `GcImpactBenchmark.java`

“ 15 benchmarks in the suite, every one mapped to a slide claim. The repo is the appendix. ”

Part 4 – Putting it together

What does a sub-10 μ s system actually look like?

Production architecture – exchange-core style



Each thread is **pinned to an isolated core**. L1/L2 stays hot. No locks. No allocations in steady state. No GC pauses.

Why all three techniques. None alone are enough.

Without...	What kills you
Off-heap	GC mark/sweep on the message buffer = unpredictable pauses
Sharding	Lock contention = cache-line ping-pong + kernel parking
Pooling	Allocation in hot path = young-gen fills = GC pause every 10 s

They're multiplicative. Each technique eliminates a different source of latency. **Skip one, and the other two can't save you.**

Production proof — exchange-core

github.com/exchange-core/exchange-core — open source

- 3M users, 10M accounts, 100K trading symbols
- Off-heap ring buffer, sharded matching engine, pooled events
- **5M ops/sec, 0.5 μ s median, p99 $\approx$ 42 μ s** on a 2014 Xeon
- Apache 2.0. Read it. Run it. Steal from it.

“ *Every technique in this talk is in that codebase. If you remember nothing else, remember the URL.* ”

What you do Monday morning

1. **Measure first.** Turn on `-Xlog:gc*:file=gc.log`. Run async-profiler. Stare at p99, not average.
2. **Find your biggest leak.** Allocation-driven GC pauses? Pool the hot path. Lock contention? Shard. CPU underutilised at high load? Lock contention is hiding.
3. **Apply ONE technique. Measure again.** Don't apply all three at once — you won't know what helped.
4. **Test at 3× peak load.** Markets get volatile. Your pool needs to be sized for that.
5. **Warm up before you go live.** A million synthetic ops triggers C2. Don't measure interpreter latency.

“ *"Don't do what I did. Don't spend a week optimising the wrong thing."* ”

Resources

Code & benchmarks:

- This talk's repo: github.com/cefothe/devoxx-uk-2026
- exchange-core: github.com/exchange-core/exchange-core
- Agrona: github.com/real-logic/agrona
- LMAX Disruptor: lmax-exchange.github.io/disruptor
- Chronicle Queue / Map: chronicle.software
- OpenHFT Affinity: github.com/OpenHFT/Java-Thread-Affinity



Further reading

- *Java Performance, 2nd ed.* — Scott Oaks (O'Reilly)
- *Mechanical Sympathy* blog — Martin Thompson
- *Achieving microsecond latencies with Java: practical techniques for building ultra-fast systems* — codebefore.eu/blog/achieving-microsecond-latencies-with-java

Thank you.

Questions?

Stefan Angelov · [linkedin.com/in/stangangelov](https://www.linkedin.com/in/stangangelov) · github.com/cefothe



“ *"The language isn't the limitation. Your understanding of the platform is."* ”